

Team Members: Nate Heath, Elliot Warner, William Thomas

Project Title: Data-Day

Goal Statement: To build and open source a 64 x 32 pixel desktop display and the developer platform for creating applications for the device.

Components: This document will detail the Device Application and Upload Server components of the D0 high level system. The Dial Button, Orange Pi, and Display components are deferred to D2.

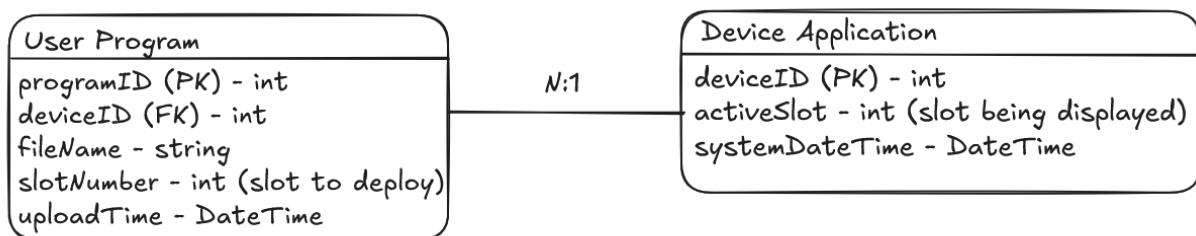
Component 1 – Device Application

This is the main application that is always running on the machine and will “host” the user-submitted applications. The main purpose of this application is to create a stable environment for the user applications to run within and allow the users to cycle between different uploaded applications.

Component 2 – Upload Server

This is the server that is hosted by the Orange Pi and will allow users to upload their custom applications from their local machine. The upload server will parse the program first before submitting it to the server, checking for malicious code. Once on the server, the Orange Pi will download the program and run it.

Data Model



Entity vs Attribute: The User Program is an entity and not an attribute of the Device Application because each uploaded program has its own metadata and must be managed and executed by the hardware.

Relationship Cardinality: The relationship is N:1 (many User Programs to 1 Device Application) because a single Orange Pi will be able to host multiple uploaded programs, but each program file is scoped to the device that it was uploaded to.

Indexed Fields: The slotNumber field on the User Program table needs to be indexed. When the user turns the dial to change the active slot, the Device Application needs a way to retrieve the matching program while minimizing delay. The deviceId should also be indexed to ensure fast joins or filtered queries upon booting up.

Relational vs. Simpler Store: For this, we will use a simpler local relations store hosted directly on the Orange Pi since the data model only needs a small list of uploaded files to the devices rotational slots. A full database would consume too much power/memory.

Core Algorithm

What: AST-based rule scanning. The main algorithm that the components listed above will contain is the algorithm to flag and remove malicious code. If slow, the program upload from the client machine to the hosted server will take a frustrating amount of time. If wrong, the algorithm will flag and remove code that is not malicious code, therefore breaking the behavior of the uploaded program. The algorithm at a high level will scan each line of the source file and check against pre-determined suspicious patterns. If it comes across any, it will flag the program as modified and remove that line. At the end, it will write the remaining lines to a new source file.

Inputs: The uploaded Python source file, as a string.

Outputs: A boolean signifying whether the uploaded code was modified, along with the possibly modified code.

Complexity: $O(nk)$, where n is the number of lines in the source file and k is the number of suspicious patterns to check for. This can be simplified to $O(n)$ as k will be constant.

Why: AST-based rule scanning is the best algorithm for a simple, quick, reliable code checker. Other possibilities are creating and training a machine learning algorithm, but we determined this is much more complicated and requires training data. When considering the low impact of letting malicious code through (the “hacker” has no one to hurt with this malicious code but themselves), we determined creating a machine learning algorithm exceeds the effort required to implement this code-checking feature. We also considered using an existing static analysis/security tool. This would work well, but cuts against the grain of the point of our project. We are trying to open-source a machine, so building a larger reliance on external resources and tools contradicts that. Finally, we considered a very simple approach like string scanning, but this is subject to lots of false positives and negatives. Since removing lines can break programs, reducing false positives is extremely important.

Edge Cases:

- Aliasing imports: Python allows you to *import subprocess as sp*, then call functions on *sp*. This means we likely have to track imports, but this would also make our algorithm more complicated.
- Indirect imports: Python allows for indirect imports (ex: *module = importlib.import_module("os")*). The scanner might not be able to detect that *module* is actually *os*.
- Code hidden inside of strings
- Dangerous network operations
- Code generated at runtime

Build-vs-Reuse Decisions

What	Build/Reuse?	Library/Service	License	Reasoning
Program Upload Server Backend	Reuse	FastAPI	MIT License	Building our own server from scratch bloats the amount of work necessary for a simple server because we would need to manually define functionality for boilerplate functionality like request parsing, protocol details, etc.
Security Algorithm	Build			Building a malicious code removal algorithm from scratch allows us more control over what code we deem as malicious and how to handle it, plus it does not add much effort.
Program Upload Frontend	Reuse	React	MIT License	Using a frontend framework such as React will speed up and simplify the process of building out the frontend to upload programs, and there is no reason not to use one.

API Contract

Endpoint Name: POST /api/v1/programs/upload

Component: Upload Server

Description: Accepts a Python file from the local machine, sends it to get sanitized, and then stores it on the Orange Pi device (if it is considered safe).

Inputs:

file (multipart/form-data): The Python file to upload

Required: Yes

Valid Range: File size must be greater than 0 bytes and strictly less than 5 MB, the file extension must be exactly .py

Outputs:

A JSON object representing the execution status and metadata of the uploaded file.

status (int): Indicates the upload result

programID (int): The primary key for the program

fileName (string): The name of the Python file

slotNumber (int): The index slot assigned for this program to deploy to

sanitizationPassed (boolean): true if the scanner found no malicious patterns, false otherwise

uploadTime (DateTime): The date and time the file was saved

Error Responses:

400 Bad Request: Returned if the file is missing, the file uploaded is not a Python file, or if sanitizationPassed is false.

500 Internal Server Error: Returned if the Orange Pi runs into a hardware/OS failure and can't save the file.

Example Request:

POST /api/v1/programs/upload HTTP/1.1

Host: orangepi.local:8000

Content-Type: multipart/form-data; boundary=----

WebKitFormBoundary7MA4YWxkTrZu0gW

-----WebKitFormBoundary7MA4YWxkTrZu0gW

Content-Disposition: form-data; name="file"; filename="weather_tracking.py"

Content-Type: text/x-python

import time

User's Python app code here...

-----WebKitFormBoundary7MA4YWxkTrZu0gW--

Example Response:

```
{  
  "status": "success",  
  "programID": 104,
```

```
"fileName": "weather_tracking.py",  
"slotNumber": 2,  
"sanitizationPassed": true,  
"uploadTime": "2026-10-08T19:35:00Z"  
}
```

Versioning Statement;

The API is versioned via the URI path (/v1/). A breaking change that requires a major version bump to /v2/ could be something like altering the structure or data types of the returned JSON output.

Technology Choices

Supported Program Language(s)

Python and C++. C++ is often used with pi devices because it is lightweight, well documented, and fast. Because of this a lot of the software for operating the Hub75 board and other attachments will be done in C++. Python is one of the simplest and most popular languages in the world. Because of this app development will be supported in Python for ease of creation.

We considered only using C++ for all development but decided against it because we want this to be as accessible as possible and python is so easy and used that it encourages people to build.

Server Network Protocol

All calls to our server should be through the API. The users will call our server via an API.

Server Host

We will be hosting the server either via a free service like Render and Github pages or Vercel. We are also considering making all of the apps and software downloadable through a standard Github repo forgoing all use of an external server.

Server Frontend

The server frontend will be made using React. This is one of the leading technologies right now, and it has many integrations and documentation.

We had briefly considered using Angular which is another popular frontend that we all have experience in, but we decided that React is a good tool to learn and easy enough for this project.